

Computer Architecture Labs

RISC-V Experiments Using the Sipeed Tang Nano 20K FPGA

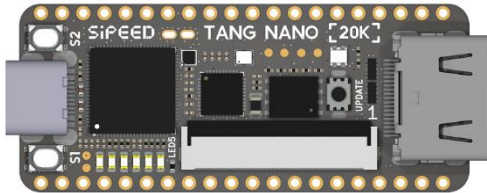


Table of Contents

1. Introduction
 2. Course Objectives
 3. Required Preparations
 4. Lab Schedule and Descriptions
 - 4.1. Lab 0 – Pre-Lab Setup: Unboxing and Environment Installation
 - 4.2. Lab 1 – Verilog Programming & LED Blinking Experiment
 - 4.3. Lab 2 – NES Emulator on FPGA
 - 4.4. Lab 3 – SparrowRV RISC-V Processor Experiment
 5. Group Work and Deliverables
 6. Assessment and Grading
 7. References and Resources
-

1. Introduction

This course is designed to introduce sophomore computer architecture students to FPGA development and RISC-V architecture using the Sipeed Tang Nano 20K FPGA development board. Over the duration of this lab course, students will learn the fundamentals of hardware description languages (HDL) with Verilog, FPGA design and debugging processes, version control using Git, and deep-dive into a practical implementation of an NES emulator and a RISC-V processor. The labs emphasize hands-on skills, source code reading, and system-level design to provide a comprehensive understanding of modern embedded systems.

2. Course Objectives

By the end of this lab course, students will be able to:

- Understand the basic operation and architecture of FPGAs.

- Write and simulate Verilog code to design digital circuits (e.g., LED blinking).
 - Utilize the high-level FPGA development environment provided by Gowin Semiconductor.
 - Employ Git version control tools to manage and collaborate on FPGA projects.
 - Interpret, compile, and deploy complex FPGA projects such as an NES emulator.
 - Analyze and understand the implementation of the RISC-V instruction set on FPGA through the SparrowRV processor.
 - Demonstrate successful integration of FPGA, peripheral hardware, and external devices.
 - Produce comprehensive lab reports articulating design choices, experimental results, and lessons learned.
-

3. Required Preparations

Prior to beginning the labs, students must complete the following setup tasks:

- **Device Unboxing and Initial Test:**
Review the “Unbox” documentation available at:
[Unboxing Tang Nano 20K FPGA Board](#)
 - **Install FPGA Development Environment:**
Download and install the Gowin FPGA development tools following the instructions provided:
[Install the IDE](#)
 - **Familiarize with Git and Version Control Principles:**
Review the “GitGuide” repository for Git basics and best practices:
[GitGuide by coderit666](#)
-

4. Lab Schedule and Descriptions

4.1 Lab 0: Pre-Lab Setup: Unboxing and Environment Installation

Description:

Students will perform an initial test of the Tang Nano 20K FPGA board by unboxing it and verifying its functionality. They will also install and set up the necessary FPGA development tools.

Goals:

- Ensure proper hardware functionality through unboxing procedures.
- Install and validate the Gowin FPGA IDE on their systems.
- Set up Git for version control as a collaborative tool for the lab course.

Deliverables:

- A brief report summarizing the hardware inspection, initial power-on tests (e.g., LED running in a “flowing” pattern), and installation screenshots of the FPGA IDE.
-

4.2 Lab 1: Verilog Programming & LED Blinking Experiment

Description:

This lab introduces Verilog fundamentals and the FPGA development process. Students will write Verilog code to create an LED blinking experiment on the Tang Nano 20K board.

Objectives:

- Learn basic Verilog syntax and coding styles.
- Understand and implement a simple counter-based LED blinking design.
- Use the Gowin FPGA IDE to compile, synthesize, and program the FPGA.
- Gain proficiency in using Git to manage source code and track changes.

Procedure:

1. **Creating a New Project:**
 - Start a new Verilog project in the FPGA IDE and set the chip model to GW2AR-LV18QN88C8/I7.
2. **Coding:**
 - Develop code for a counter that toggles an LED every 0.5 seconds.
 - Include comments and proper module structure.
3. **Synthesis and Implementation:**
 - Run synthesis and floor-plan tools to map your Verilog code to the FPGA's resources.
4. **Programming:**
 - Use the integrated Programmer tool to load the compiled file (.fs) into the FPGA's memory.
5. **Version Control:**
 - Commit the project source to a local Git repository and push to a remote Git repository for backup and collaboration.

Deliverables:

- Source code file(s) with detailed comments.
- A demonstration (video or live demonstration) of the LED blinking functionality.
- A lab report documenting the design, synthesis process, challenges encountered, and Git usage.

Reference Material:

- Verilog Tutorials <https://www.w3cschool.cn/verilog/verilog-rdio3o3b.html>
 - GitGuide repository
-

4.3 Lab 2: NES Emulator on FPGA

Description:

Students will work with an existing NES emulator project for the Tang Nano 20K board. This lab requires reading and understanding the source code, setting up peripherals (joysticks, HDMI monitor, TF card), and achieving full compilation and execution.

Objectives:

- Understand the interaction between the FPGA and external devices (display, input controllers, storage).
- Analyze the NES architecture and instruction set principles.

- Compile and deploy an NES emulator on the FPGA board.
- Evaluate system performance and troubleshoot hardware–software interfacing issues.

Procedure:

1. **Source Code Retrieval:**
 - Clone the NES emulator repository from:
[GitHub NES Emulator for Tang Nano 20K](#)
2. **Hardware Setup:**
 - Connect external devices: one HDMI monitor, two joystick convert boards, a TF card (pre-programmed with game images).
3. **Compilation and Deployment:**
 - Follow the provided guidelines to compile and burn the firmware into the FPGA's flash memory using tools such as Balena Etcher or dd.
4. **Source Code Analysis:**
 - Study the structure of the NES emulator, focusing on its architecture, instruction set handling, and peripheral interaction.
5. **Testing:**
 - Run the emulator and select games from the menu. Document any issues and discuss potential improvements.

Deliverables:

- Laboratory demonstration (record a short video or provide detailed screenshots) showing the emulator in action.
- A comprehensive lab report including an analysis of the NES architecture and your understanding of the codebase.

Reference Material:

- NES architecture overview: [NES Architecture Resource](#)
 - NES emulator Github repository
-

4.4 Lab 3: SparrowRV RISC-V Processor Experiment

Description:

In this lab, students will explore the implementation of the RISC-V instruction set on FPGA by working with the SparrowRV processor project. The task involves reading the source code, compiling the processor design, and running a simple “Hello World” program.

Objectives:

- Understand the principles behind implementing a RISC-V compatible processor on an FPGA.
- Analyze the SparrowRV processor's source code to grasp the core concepts of pipelined processing and instruction set architecture.
- Successfully compile and program the FPGA with the SparrowRV design.
- Execute a “Hello World” test on the running processor.

Procedure:

1. **Code Acquisition:**

- Clone the project from the SparrowRV repository:
[SparrowRV GitHub Repository](#) or [Gitee Repository \(Latest\)](#)
- 2. **Code Analysis:**
 - In groups, review the design documents and source code to identify pipeline stages, control logic, and bus interfaces.
- 3. **Compilation and Deployment:**
 - Set up your project in the FPGA IDE and compile the SparrowRV design.
 - Program the FPGA with the compiled design.
- 4. **Testing:**
 - Load and execute a “Hello World” program on the SparrowRV processor.
 - Verify functionality through serial output or on-board LEDs.
- 5. **Troubleshooting:**
 - Document any issues encountered, methods used to diagnose them, and solutions.

Deliverables:

- Source code and synthesis log files.
- A demonstration (video recording or live demo) of the “Hello World” program running.
- A detailed lab report outlining the design insights, challenges overcome, and learnings from the SparrowRV processor project.

Reference Material:

- SparrowRV Project Repository
 - RISC-V Architecture Documentation
-

5. Group Work and Deliverables

- **Team Formation:** Students shall form **teams of two or three**.
 - **Lab Reports:** Each team must submit one single comprehensive lab report including all the experiments.
The lab report should include:
 - Objectives and methodology.
 - Detailed design and implementation steps.
 - Screenshots/logs from synthesis, programming, and testing.
 - Discussion on challenges, troubleshooting efforts, and conclusions.
 - **Extra Credit:** Teams that document innovative solutions or include additional research insights may receive extra credit.
-

6. Assessment and Grading

Grading will be based on the following components:

- **Pre-Lab Preparation:** (10%) – Verification of device unboxing and environment setup.
- **Lab Execution:** (40%) – Successful implementation and demonstration of each

lab.

- **Lab Reports:** (40%) – Clarity, depth, technical accuracy, and presentation.
 - **Team Collaboration and Participation:** (10%) – Contribution and teamwork, as assessed through peer reviews.
-

7. References and Resources

- Sipeed Tang Nano 20K FPGA Board Information:
[20K Board Overview](#)
- Unboxing Instructions:
[Unboxing Tang Nano 20K](#)
- FPGA IDE Installation:
[Install the IDE](#)
- Git Basic Tutorial:
[GitGuide by coderit666](#)
- NES Emulator Project:
[GitHub NES Emulator Example](#)
- NES System Architecture Overview:
[NES Architecture Resource](#)
- SparrowRV RISC-V Processor:
[GitHub SparrowRV Repository](#)